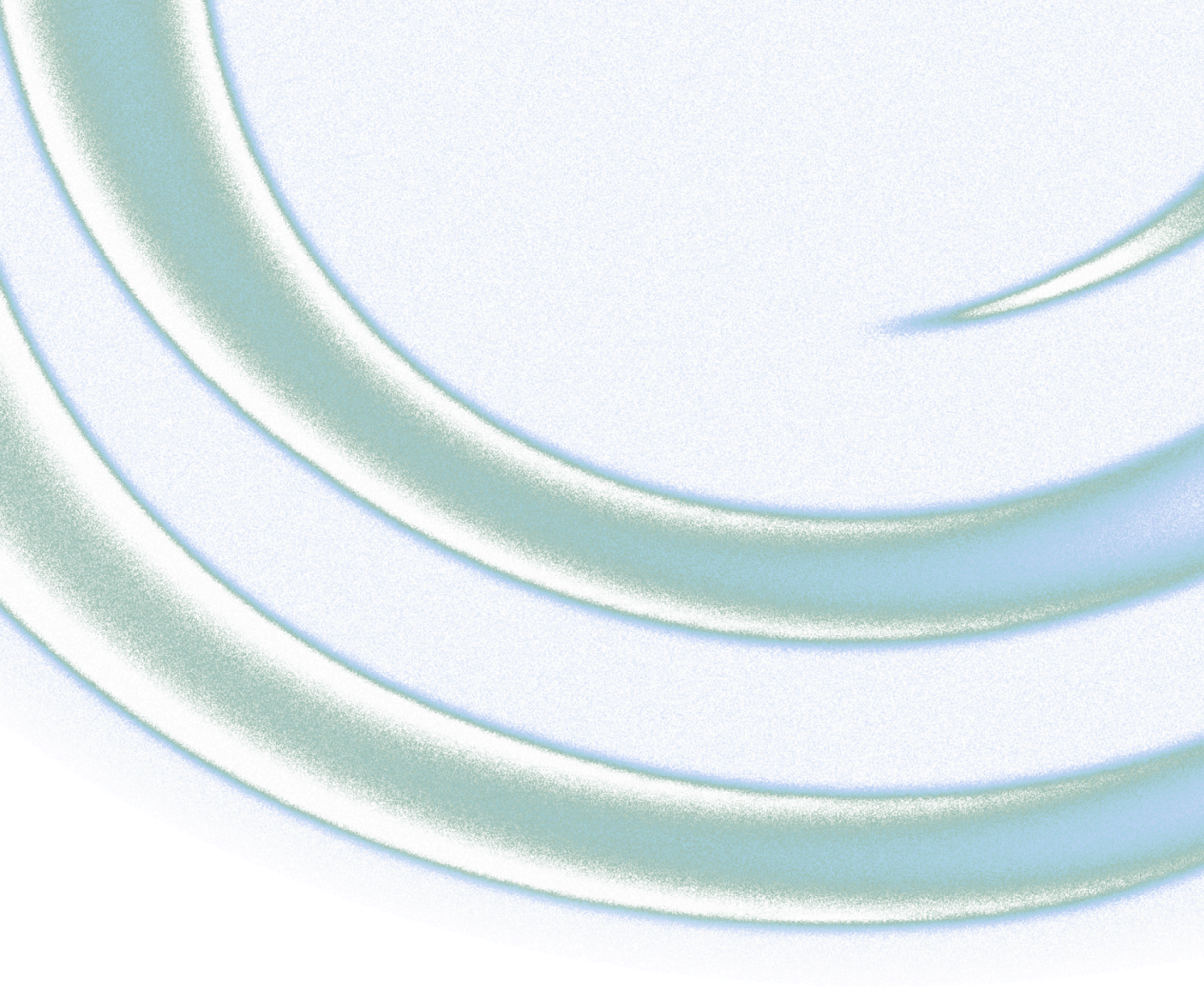


OtterSec

Multi Host Tenant Solana

Security Assessment

Prepared for
Wormhole Foundation

Prepared by
Otter Audits, LLC

Prepared on
August 18th, 2026

Gabriel Ottoboni
ottoboni@osec.io

Julian K
julian@osec.io

Table of Contents

1. Executive Summary

1.1. Overview

Wormhole Foundation engaged OtterSec to conduct a security assessment of the `multi-tenant deployment` PR. This assessment was conducted between July 27th and August 1st, 2026. For information on our auditing methodology, refer to Appendix B.

1.2. Key Findings

We produced 3 findings throughout this audit engagement.

Among the most impactful results, we identified that peer rotation may block pending VAA redemption (● OS-MHT-ADV-00). We also advised removing outdated comments and unused code (● OS-MHT-SUG-00). Lastly, we suggested improving error handling (● OS-MHT-SUG-01).

2. Scope

PR#875 upgrades Solana NTT to a multi-instance architecture, allowing one program to host multiple isolated deployments identified by separate Config accounts. It scopes all state/PDAs and cross-chain manager identities to each Config while separating instance ownership from program upgrade authority.

The source code was delivered to us in a Git repository at `https://github.com/wormhole-foundation/native-token-transfers`. This audit was performed against PR#875.

3. Findings

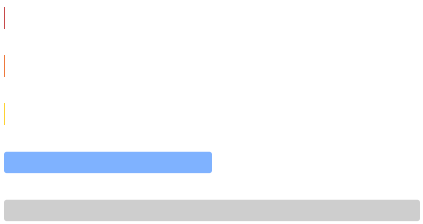
Findings were rated using the criteria in Appendix A.

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: `OS-MHT-ADV-*` for advisories and `OS-MHT-SUG-*` for suggestions.

3.1. Findings summary

Severity	Count
Critical	0
High	0
Medium	0
Low	1
Informational	2



4. Advisories

Here, we present a technical analysis of the 1 vulnerability we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

ID	Severity	Status	Description
OS-MHT-ADV-00	Low	Ack'd	Peer Rotation Blocks Pending VAA Redemption

Peer Rotation Blocks Pending VAA Redemption

ID	Severity	Status
OS-MHT-ADV-00	Low	Ack'd

Description

`set_peer` overwrites the existing `NttManagerPeer` because its PDA is keyed only by `config + chain_id`, not by the peer address. After rotation, the account remains at the same PDA but contains the new remote manager address. Any older VAA still requiring `redeem` carries the previous `source_ntt_manager` and therefore fails the `peer.address == source_ntt_manager` constraint. This can permanently block pending messages or additional transceiver votes.

solana/programs/example-native-token-transfers/src/instructions/redeem.rs

```
#[derive(Accounts)]
pub struct Redeem<'info> {
    [...]
    #[account(
        seeds = [NttManagerPeer::SEED_PREFIX, config.key().as_ref(),
        ValidatedTransceiverMessage::<NativeTokenTransfer<Payload>>::from_chain(&transcei
        constraint = peer.address ==
        ValidatedTransceiverMessage::<NativeTokenTransfer<Payload>>::message(&transceiver
        [..])?.source_ntt_manager() @ NTTError::InvalidNttManagerPeer,
        bump = peer.bump,
    )]
    pub peer: Account<'info, NttManagerPeer>,
    [...]
}
```

Remediation

Preserve old peer addresses during a configurable grace period so pending VAAs remain redeemable after peer rotation.

Patch

Wormhole Foundation acknowledged this issue.

5. Suggestions

Here, we present a discussion of the 2 suggestions we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-MHT-SUG-00	Informational	Ack'd	Code Maturity
OS-MHT-SUG-01	Informational	Ack'd	Improper Error Handling

Code Maturity

ID	Severity	Status
OS-MHT-SUG-00	Informational	Ack'd

Description

There are multiple instances of unnecessary code in the current version that may be removed for improved readability.

Firstly, in v4, multiple `OutboxRateLimit` accounts can exist under the same program—one per Config-scoped NTT instance. The comment is therefore misleading because instructions must validate the Config-seeded PDA to prevent cross-instance account substitution. Ensure to remove this outdated comment.

solana/programs/example-native-token-transfers/src/queue/outbox.rs

```
/// Global rate limit for all outbound transfers to all chains.
/// NOTE: only one of this account can exist, so we don't need to check the PDA.
impl OutboxRateLimit {
    pub const SEED_PREFIX: &'static [u8] = b"outbox_rate_limit";
}
```

Also, the `InvalidDeployer` error variant is no longer used and should be removed.

solana/programs/example-native-token-transfers/src/error.rs

```
#[error_code]
// TODO(csongor): rename
#[derive(PartialEq)]
pub enum NTTError {
    [...]
    #[msg("InvalidDeployer")]
    InvalidDeployer,
    [...]
}
```

Remediation

Remove the outdated comment and unused code instance.

Patch

Wormhole Foundation acknowledged this recommendation.

Improper Error Handling

ID	Severity	Status
OS-MHT-SUG-01	Informational	Ack'd

Description

In the current implementation, the two `unwrap()` calls in `from_chain` can panic on a borrow conflict or malformed account data, respectively, allowing invalid input to abort execution instead of returning an Anchor error. Thus, it will be appropriate to replace `.unwrap()` with `?` to handle errors more gracefully, making `from_chain` fully fallible and consistent with its `Result<ChainId>` return type.

solana/programs/example-native-token-transfers/src/messages.rs

```
1  pub fn from_chain(info: &UncheckedAccount) -> Result<ChainId> {
2      let data: &[u8] = &info.try_borrow_data().unwrap();
3      Self::discriminator_check(data)?;
4      Ok(ChainId {
5          // This is LE bytes because we deserialize using Borsh.
6          // Not to be confused with the wire format (which is BE bytes)
7          id: u16::from_le_bytes(data[8..10].try_into().unwrap()),
8      })
9  }
```

Remediation

Replace the `.unwrap()` calls with `?` in `from_chain`.

Patch

Wormhole Foundation acknowledged this recommendation.

Appendix A.

Vulnerability rating scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of ✓ Resolved, ○ Ack'd, or × To-do.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more checklist'' style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process